FIAP

Introdução às Matrizes em C

Ciência da Computação

Prof. Dr. Érick Toshio Yamamoto

Objetivos da Aula - Parte 1

FIAP

O que vamos aprender hoje:

- Compreender a **transição lógica** de vetores para matrizes.
- Dominar a **declaração e inicialização** de arrays multidimensionais.
- Entender a **organização física** de matrizes na memória RAM.
- Praticar o **acesso a elementos** usando índices de linha e coluna.

Fundamentos: Construindo a Base para Estruturas de Dados Complexas

Objetivos da Aula - Parte 2

FIAP

O que vamos aprender hoje:

- Revisitando o conceito de **Strings** como vetores de caracteres.
- Compreender o conceito de **Matriz de Strings** (Vetor de Nomes).
- Entender a organização de **nomes na memória RAM** e o terminador nulo.
- Praticar a **leitura e exibição** de listas de nomes em C.

Foco: Gestão de Dados Textuais em Estruturas Multidimensionais

Recapitulação: O que é um Vetor?

FIAP

- Um vetor é uma **lista sequencial** de dados do mesmo tipo.
- Possui apenas **uma dimensão** (tamanho definido).
- Acesso via **índice único** : vetor[i] .

> ***Exemplo:*** *int notas[10]; armazena 10 notas em uma única fileira na memória.*

```
int notas[5] = {10, 8, 9, 7, 6};
```

Representação Visual:



A Evolução: Por que Matrizes?

FIAP

- Vetores são ótimos para **listas simples** de dados sequenciais.
- Matrizes são necessárias para dados organizados em **grade ou tabela**.
- Uma matriz é, essencialmente, um "**vetor de vetores**".

> ***Conceito:*** *Quando uma única dimensão não é suficiente para representar a relação entre os dados.*

Exemplos do Mundo Real:

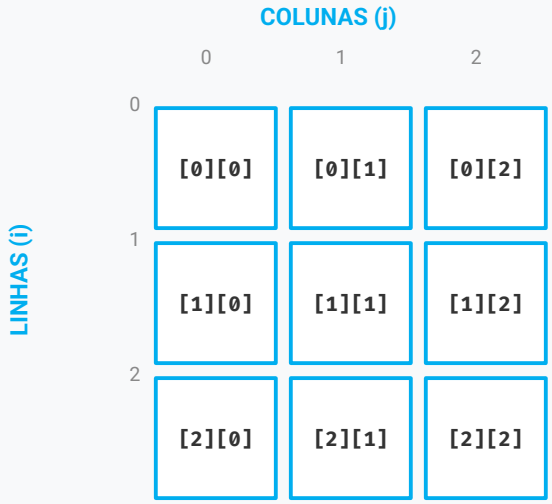
- **Planilhas de Excel (Linhas e Colunas)**
- **Tabuleiros de Jogos (Xadrez, Batalha Naval)**
- **Pixels de uma Imagem Digital**
- **Mapa de Assentos (Cinema, Teatro, Avião)**

Visualizando a Matriz Lógica

FIAP

- Uma matriz é uma **grade bidimensional** composta por Linhas e Colunas.
- Cada célula é identificada por um par de coordenadas: **(linha, coluna)**.
- **Indexação:** Assim como nos vetores, a contagem sempre começa em **0**.

> ***Analogia:*** *Pense em uma planilha de Excel, onde as linhas são números e as colunas são letras.*



Declaração de Matrizes em C

FIAP

Sintaxe:
tipo nome[linhas][colunas];

Dimensões: O primeiro colchete define as **linhas** e o segundo as **colunas**.

Capacidade: O total de elementos é o produto das dimensões (ex: 4x5 = 20).

> ***Regra:*** *Os tamanhos devem ser valores inteiros constantes e positivos.*

```c
// Matriz de inteiros (4x5)
int tabela[4][5];
 // Matriz de reais (3x3)
float notas[3][3];
 // Matriz de caracteres (2x10)
char nomes[2][10];
```

Inicialização de Matrizes

FIAP

- **Inicialização Direta:** Usamos chaves aninhadas para definir cada linha.
- **Zerar Matriz:** O atalho `{0}` preenche todos os elementos com zero.
- **Parcial:** Se faltarem valores, o C preenche o restante com zero automaticamente.

> ***Dica:*** *Organize os valores em linhas no código para facilitar a leitura humana.*

```c
// Inicializacao completa (2x2)
int mat[2][2] = {
{1, 2}, // Linha 0
{3, 4} // Linha 1
};
 // Zerando toda a matriz
int zeros[5][5] = {0};
 // Inicializacao parcial
int p[2][2] = {{1}, {3}};
// p[0][1] e p[1][1] serao 0
```

Matrizes na Memória RAM

FIAP

Alocação Contígua: Na memória RAM, a matriz é armazenada como uma única linha gigante.

Row-Major Order: O C armazena primeiro todos os elementos da linha 0, depois da linha 1, e assim por diante.

Performance: Acessar a memória sequencialmente é muito mais rápido para o processador (Cache).

> ***Importante:*** *O computador "achata" a matriz para otimizar o acesso físico.*

Visão Lógica (Tabela 2x3)

0,0	0,1	0,2
1,0	1,1	1,2

Visão Física (Memória RAM Linear)



O C garante que elementos da mesma linha estejam vizinhos na memória.

Acesso e Atribuição de Dados

FIAP

- Para manipular um dado, usamos **dois colchetes** `matriz[L][C]` .
- **Regra de Ouro:** Os índices variam de **0** até **N-1**.
- **Cuidado:** Acessar `matriz[5][10]` em uma matriz 5x10 causará erro de memória.

> ***Atenção:*** *O primeiro índice é sempre a Linha e o segundo é sempre a Coluna.*

```c
int mat[3][3];
 // Atribuindo valor a uma posicao
mat[0][1] = 10;
 // Lendo valor do teclado
scanf("%d", &mat[2][2]);
 // Exibindo um valor especifico
printf("Valor: %d", mat[0][1]);
```

Percorrendo Matrizes com For Aninhado

FIAP

- Para navegar em duas dimensões, precisamos de um **laço dentro de outro**.
- O **laço externo** controla as **linhas** (índice `i`).
- O **laço interno** controla as **colunas** (índice `j`).

> ***Lógica:*** *Para cada linha `i`, o laço interno percorre todas as colunas `j` antes de avançar para a próxima linha.*

```c
int i, j;
int mat[3][3];
 // Percorrendo a matriz para exibicao
for(i = 0; i < 3; i++) {
       for(j = 0; j < 3; j++) {
            printf("%d ", mat[i][j]);
       }
        printf("\n"); // Pula linha apos cada linha
da matriz
}
```

Revisando Strings: O Vetor de Char

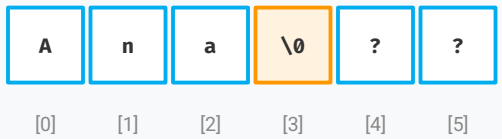
FIAP

- Uma string em C é um **vetor de caracteres** (char).
- O caractere especial **\0** (nulo) indica o fim da string na memória.
- Declaração: char nome[20]; reserva espaço para 19 letras + o \0.

> ***Conceito:*** *Uma string é unidimensional. Mas e se precisarmos de uma* ***lista*** *de nomes?*

```
char nome[6] = "Ana";
```

Representação na Memória:



\0 = Terminador Nulo

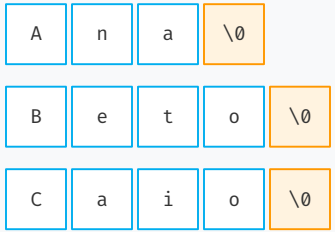
O Conceito de Matriz de Strings

FIAP

- Uma matriz de strings é um **vetor** onde cada elemento é uma **string** independente.
- **Estrutura:** A primeira dimensão define a **quantidade de nomes**.
- A segunda dimensão define o **tamanho máximo** de cada nome.

> ***Analogia:*** *Pense em uma lista de chamada, onde cada linha contém o nome de um aluno diferente.*

Exemplo: Lista de 3 Nomes



Cada linha é uma string completa com seu próprio terminador.

Visualizando Nomes na Memória RAM

FIAP

Cada **linha** da matriz armazena uma string independente.

O caractere **\0** é obrigatório no final de cada nome para delimitar o fim da string.

Espaços não utilizados após o \0 contêm "lixo de memória" ou zeros.

Estrutura: *Uma matriz* *char lista[3][6]* *pode guardar 3 nomes de até 5 letras cada.*



O terminador nulo **\0** permite que funções como printf("%s") saibam onde parar.

Declaração e Inicialização de Nomes

FIAP

Sintaxe: char lista[QNT][TAM];

Inicialização: O C coloca o \0 automaticamente no final de cada string entre aspas.

Atenção: O tamanho da 2ª dimensão deve comportar o maior nome + 1 (para o \0).

> ***Dica:*** *Use constantes (#define) para definir os tamanhos e facilitar a manutenção.*

```c
// Lista de 3 cores, cada uma com ate 9 letras
char cores[3][10] = {
"Verde",
"Azul",
"Preto"
};
 // Matriz vazia para 30 alunos
char alunos[30][50];
 // Inicializacao parcial (restante vira \0)
char dias[7][10] = {"Segunda", "Terca"};
```

Entrada de Dados: Lendo Vários Nomes

FIAP

- Usamos um laço **for** para iterar sobre as linhas da matriz.
- **Segurança:** Prefira `fgets` ao invés de `scanf` para evitar estouro de buffer.
- Cada chamada de leitura preenche uma **linha inteira** da matriz.

> ***Dica:*** *Lembre-se que o* `fgets` *também lê o caractere* `\n` *(Enter) ao final da string.*

```c
char alunos[5][50];
int i;
 for(i = 0; i < 5; i++) {
printf("Digite o nome do aluno %d: ", i+1);
// Le ate 49 caracteres + \0
fgets(alunos[i], 50, stdin);
}
```

Exibindo a Lista de Nomes na Tela

FIAP

- Para imprimir uma string da matriz, usamos apenas **UM índice**.
- O índice `i` aponta para o **início da linha** (a string completa).
- O `printf` com `%s` percorre a linha até encontrar o `\0`.

> ***Diferença:*** *`nomes[i]` é a string inteira, enquanto `nomes[i][j]` é um único caractere.*

```
char alunos[5][50];
int i;
 // Exibindo todos os nomes da lista
for(i = 0; i < 5; i++) {
printf("Aluno %d: %s\n", i+1, alunos[i]);
}
 // Exibindo apenas o primeiro caractere do
primeiro nome
printf("Inicial: %c", alunos[0][0]);
```

Manipulação com a Biblioteca string.h

FIAP

strcpy: Copia uma string para uma linha específica da matriz.

strcmp: Compara uma string externa com um nome da lista.

strlen: Retorna o tamanho real de um nome em uma linha.

Importante: *Ao usar estas funções, passamos apenas o primeiro índice:*

```
funcao(matriz[i])
```

.

```c
char nomes[10][50];
 // Copiando um nome para a linha 0
strcpy(nomes[0], "Joao Silva");
 // Comparando nomes
if(strcmp(nomes[0], "Joao Silva") == 0) {
printf("Nome encontrado!");
}
 // Obtendo o tamanho do nome
int tam = strlen(nomes[0]);
```

Exemplo Prático: Lista de Presença Inversa

FIAP

```c
#include <stdio.h>
#include <string.h>
 int main() {
    char lista[5][50];
    int i;
 // Leitura dos nomes
    for(i = 0; i < 5; i++) {
        printf("Nome %d: ", i+1);
        fgets(lista[i], 50, stdin);
        lista[i][strcspn(lista[i], "\n")] = 0;
        }
 // Exibicao em ordem inversa
    printf("\n--- LISTA INVERSA ---\n");
    for(i = 4; i >= 0; i--) {
        printf("%s\n", lista[i]);
    }
    return 0;
}
```

1. Matriz de Strings: Criamos 5 espaços de 50 caracteres cada.

2. Limpeza do \n: A função `strcspn` localiza o Enter e o substitui por `\0`, limpando a string.

3. Lógica Inversa: O segundo laço começa no índice **4** e decrementa até **0**.

💡Note que usamos apenas `lista[i]` para imprimir o nome completo.

Desafio para Casa: Cadastro de Produtos

FIAP

Pratique em casa: Sistema de Busca de Inventário

- Crie uma matriz `char produtos[10][30]` para armazenar 10 nomes de produtos.
- Desenvolva um laço para o usuário cadastrar todos os produtos.
- Após o cadastro, peça ao usuário para digitar o nome de um produto para **busca**.
- Use a função `strcmp` dentro de um laço para verificar se o nome existe na matriz.
- Se encontrar, exiba "Produto em Estoque". Caso contrário, exiba "Não Encontrado".

Dica: Lembre-se de incluir a biblioteca `<string.h>` e tratar o `\n` do `fgets` para que a comparação funcione corretamente!